**Date: 10-10-2026**

**CO-1 - AT-1**

**APPLICATION BASED QUESTIONS**

**COURES CODE: DSA0308**

**COUSRE NAME: NLP**

**Reg no: 192425034** **Name: Vinay Kandala**

## Question 1: Resume Information Extraction

Regular expressions (Regex) are used to identify patterns in text. In this program, the re module extracts the candidate's name, email address, mobile number, technical skills, and experience from the resume.

- re.search() identifies the candidate's name and experience.
- re.findall() extracts all matching email addresses and mobile numbers.
- re.escape() protects special characters when constructing a regular expression dynamically.
- The program checks for Python skills and at least two years of experience.
- A candidate is considered eligible only if both conditions are satisfied

## Python code

```
import re

SKILLS = ["Python", "Java", "SQL", "Machine Learning", "NLP"]


def extract_resume(text):
    name = re.search(r"(?im)^\s*Name\s*:\s*([A-Za-z][A-Za-z .'-]*)\s*$", text)
    emails = re.findall(r"\b[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}\b", text)
    mobiles = re.findall(r"(?<!\d)(?:\+91[\s-]?)?[6-9]\d{9}(?!\d)", text)
    exp = re.search(r"(?i)\bExperience\s*:\s*(\d+)\s*\+?\s*years?\b", text)
```

```
    if not exp:
        exp = re.search(r"(?i)\b(\d+)\s*\+?\s*years?\s+(?:of\s+)?experience\b", text)
    skills = [s for s in SKILLS if re.search(r"(?<!\w)" + re.escape(s) + r"(?!\w)", text, re.I)]
    years = int(exp.group(1)) if exp else None
    eligible = years is not None and years >= 2 and "Python" in skills
    return {
        "Name": name.group(1).strip() if name else "Not found",
        "Email": emails,
        "Mobile": mobiles,
        "Skills": skills,
        "Experience": years if years is not None else "Not found",
        "Eligible": eligible
    }

def main():
    resume = """Name: Vinay Kandala
Email: vinay.kandala@example.com
Mobile: 9876543210
Skills: Python, SQL, NLP
Experience: 3 years"""
    result = extract resume(resume)
    print("RESUME INFORMATION EXTRACTION SYSTEM")
    for key, value in result. Items():
        print(f"{key}: {value}")

if __name__ == "__main__":
    main()
```

## output

```
RESUME INFORMATION EXTRACTION SYSTEM
```

Name: Vinay Kandala

Email: ['vinay.kandala@example.com']

Mobile: ['9876543210']

Skills: ['Python', 'SQL', 'NLP']

Experience: 3

Eligible: True

## Question 2: Product Search System

Regular expressions allow the program to search a product list using different matching conditions. The program stores product names in a list and compares each product with the keyword provided.

- Exact search: Finds a keyword as a complete word.
- Prefix search: Finds product names that contain the keyword at the beginning of a word.
- Suffix search: Finds product names containing a word ending with the specified keyword.
- Partial search: Finds the keyword anywhere within a product name.
- Case-insensitive search: Matches the keyword regardless of uppercase or lowercase letters.
- The program displays matching products and counts the total number of matches.

## Python Code

```
import re


products = [
    "Apple iPhone 15",
    "Samsung Galaxy S24",
    "Apple MacBook Air",
    "Dell Inspiron Laptop",
```

```
    "HP Pavilion Laptop",
    "Samsung Galaxy Buds",
    "Logitech Wireless Mouse",
    "Apple Watch Series 9",
    "Sony Wireless Headphones",
    "OnePlus Nord Phone"
]

def search_products(keyword, mode):
    if not keyword.strip():
        matches = []
    else:
        keyword = re.escape(keyword)

        if mode == "exact":
            pattern = r"(?<!\w)" + keyword + r"(?!\w)"
        elif mode == "prefix":
            pattern = r"\b" + keyword
        elif mode == "suffix":
            pattern = keyword + r"\b"
        elif mode in ("partial", "case-insensitive"):
            pattern = keyword
        else:
            print("Invalid search mode")
            return

        matches = [
            product for product in products
            if re.search(pattern, product, re.I)
```

```
        ]

    print(f"Candidate: Vinay Kandala | Mode: {mode}")
    print("Keyword:", keyword if keyword.strip() else "(empty)")
    print("Matching products:")

    for product in matches:
        print("-", product)

    print("Total matches:", len(matches))
    print("-" * 35)


print("PRODUCT SEARCH SYSTEM")

search_products("Apple", "exact")
search_products("Sam", "prefix")
search_products("top", "suffix")
search_products("wireless", "partial")
search_products("APPLE", "case-insensitive")
```

## output

```
PRODUCT SEARCH SYSTEM

Candidate: Vinay Kandala | Mode: exact
Keyword: Apple
Matching products:
- Apple iPhone 15
- Apple MacBook Air
```

- Apple Watch Series 9

Total matches: 3

Candidate: Vinay Kandala | Mode: prefix

Keyword: Sam

Matching products:

- Samsung Galaxy S24

- Samsung Galaxy Buds

Total matches: 2

## Question 3: University Registration System

Regular expressions are used to verify whether the entered student information follows the required format. The program defines a separate pattern for each field and uses re.fullmatch() to ensure that the complete input matches the pattern.

- Register number: Checks the expected combination of digits and uppercase letters.
- Institutional email: Checks whether the email uses one of the configured institutional domains.
- Course code: Checks for uppercase letters followed by digits.
- Semester: Accepts semester numbers from 1 to 8, including the format Semester 4.
- Mobile number: Checks for a valid 10-digit Indian mobile number, optionally with the +91 country code.
- The program displays VALID or INVALID for each field.
- The final registration status is SUCCESSFUL only when every field is valid.

## Python Code

import re

```
patterns = {
    "Register Number": r"\d{2}[A-Z]{2,4}\d{3,4}",
    "Institutional Email":
        r"[A-Za-z0-9._%+-]+@(?:simats\.edu\.in|saveetha\.com)",
    "Course Code": r"[A-Z]{2,5}\d{2,4}",
    "Semester": r"(?:[1-8]|Semester\s+[1-8])",
    "Mobile Number": r"(?:\+91[\s-]?)?[6-9]\d{9}"
}

def validate_registration(reg_no, email, course, semester, mobile):
    values = {
        "Register Number": reg_no,
        "Institutional Email": email,
        "Course Code": course,
        "Semester": semester,
        "Mobile Number": mobile
    }

    all_valid = True

    for field, value in values.items():
        valid = re.fullmatch(
            patterns[field], value.strip(), re.I
        ) is not None

        print(f"{field}: {'VALID' if valid else 'INVALID'}")

        if not valid:
            all_valid = False
```

```
    status = "SUCCESSFUL" if all_valid else "UNSUCCESSFUL"
    print("Final Registration Status:", status)


print("UNIVERSITY REGISTRATION SYSTEM")
print("STUDENT 1: Vinay Kandala")

validate_registration(
    "23AIML1234",
    "vinay.kandala@simats.edu.in",
    "NLP203",
    "4",
    "9876543210"
)

print("\nSTUDENT 2: Vinay Kandala (invalid test data)")

validate_registration(
    "AB123",
    "vinay@gmail.com",
    "nlp",
    "9",
    "12345"
)
```

## Output

```
UNIVERSITY REGISTRATION SYSTEM
STUDENT 1: Vinay Kandala
```

Register Number: VALID

Institutional Email: VALID

Course Code: VALID

Semester: VALID

Mobile Number: VALID

Final Registration Status: SUCCESSFUL

STUDENT 2: Vinay Kandala (invalid test data)

Register Number: INVALID

Institutional Email: INVALID

Course Code: INVALID

Semester: INVALID

Mobile Number: INVALID

Final Registration Status: UNSUCCESSFUL